NLP Methods: Rules, Stats, & Transformers

This notebook demonstrates how to use spaCy for traditional rule-based NLP as well as modern statistical NLP using pretrained models. We'll also briefly cover the use of huggingface modules to integrate transformer-based NLP methods.

Rule-Based NLP with spaCy

In [ ]:
```
# !pip install spacy spacy-transformers
```

Linguistic Breakdowns

In [ ]:
```
import spacy
from spacy.matcher import Matcher, PhraseMatcher
```

In [2]:
```
nlp = spacy.load("en_core_web_sm")
```

In [3]:
```
text = "Apple is looking at buying a U.K. startup for $1 billion."
doc = nlp(text)

for token in doc:
    print(f"{token.text:12} | POS: {token.pos_:6} | Tag: {token.tag_:6} | Lemma: {t
```

```
Apple        | POS: PROPN  | Tag: NNP    | Lemma: Apple
is           | POS: AUX    | Tag: VBZ    | Lemma: be
looking      | POS: VERB   | Tag: VBG    | Lemma: look
at           | POS: ADP    | Tag: IN     | Lemma: at
buying       | POS: VERB   | Tag: VBG    | Lemma: buy
a            | POS: DET    | Tag: DT     | Lemma: a
U.K.         | POS: PROPN  | Tag: NNP    | Lemma: U.K.
startup      | POS: NOUN   | Tag: NN     | Lemma: startup
for          | POS: ADP    | Tag: IN     | Lemma: for
$            | POS: SYM    | Tag: $      | Lemma: $
1            | POS: NUM    | Tag: CD     | Lemma: 1
billion      | POS: NUM    | Tag: CD     | Lemma: billion
.            | POS: PUNCT  | Tag: .      | Lemma: .
```

Pattern Matching

In [4]:
```
matcher = Matcher(nlp.vocab)

pattern = [
    {"POS": "VERB"},
    {"POS": "PROPN"}
]

matcher.add("VERB_PROPN_PATTERN", [pattern])
```

```
matches = matcher(doc)

for match_id, start, end in matches:
    span = doc[start:end]
    print(span.text)
```

Phrase Matching

In [5]:
```
phrases = ["U.K.", "Apple", "startup"]
phrase_patterns = list(nlp.pipe(phrases))

phrase_matcher = PhraseMatcher(nlp.vocab)
phrase_matcher.add("COMPANY_TERMS", phrase_patterns)

matches = phrase_matcher(doc)
for match_id, start, end in matches:
    print(doc[start:end].text)
```

```
Apple
U.K.
startup
```

Statistical Analysis NLP with spaCy

In [ ]:
```
import pandas as pd
```

Named Entities | Doc Properties

In [38]:
```
doc.ents
```

Out[38]:
```
(Apple, U.K., $1 billion)
```

In [42]:
```
ent_list = []
for ent in doc.ents:
    ent_list.append({'text':ent.text, 'label':ent.label_,})
ent_df = pd.DataFrame(ent_list)
ent_df.head()
```

Out[42]:

	text	label
0	Apple	ORG
1	U.K.	GPE
2	$1 billion	MONEY

In [8]:
```
for sent in doc.sents:
    print(sent.text)
```

```
Apple is looking at buying a U.K. startup for $1 billion.
```

Word Vectors & Semantic Similarity

In [ ]:
```
nlp_md = spacy.load("en_core_web_md") # vectorizes sentences

doc1 = nlp_md("I love dogs") # mean-pooled word vector
doc2 = nlp_md("I adore puppies")
print("Similarity:", doc1.similarity(doc2)) # cosine similarity
```

```
Similarity: 0.8026599884033203
```

Extracting Linguistic Features at Token Level

In [ ]:
```
'''
head -- token upon which the current token is syntactically dependent on
dep -- how exactly the token is dependent upon the head token
'''
token_list = []
for token in doc:
    token_list.append(
        {
            'text':token.text,
            'head':token.head.text,
            'dep':token.dep_,
        }
    )
token_df = pd.DataFrame(token_list)
token_df.head(20)
```

Out[ ]:

	text	head	dep
0	Apple	looking	nsubj
1	is	looking	aux
2	looking	looking	ROOT
3	at	looking	prep
4	buying	at	pcomp
5	a	U.K.	det
6	U.K.	buying	dobj
7	startup	looking	advcl
8	for	startup	prep
9	$	billion	quantmod
10	1	billion	compound
11	billion	for	pobj
12	.	looking	punct

Huggingface Transfomers for NLP

In [14]:
```
# !pip install -q transformers torch --upgrade
```

In [22]:
```
from transformers import AutoTokenizer, AutoModel, pipeline
import torch
import pandas as pd
```

Next Token Prediction

In [ ]:
```
mlm = pipeline( # Masked Language Modeling
    task="fill-mask",
    model="bert-base-uncased", # original bert model
    tokenizer="bert-base-uncased",
    top_k=5
)

results = mlm("The capital of France is [MASK].")
```

```
Some weights of the model checkpoint at bert-base-uncased were not used when initial
izing BertForMaskedLM: ['bert.pooler.dense.bias', 'bert.pooler.dense.weight', 'cls.s
eq_relationship.bias', 'cls.seq_relationship.weight']
- This IS expected if you are initializing BertForMaskedLM from the checkpoint of a
model trained on another task or with another architecture (e.g. initializing a Bert
ForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing BertForMaskedLM from the checkpoint o
f a model that you expect to be exactly identical (initializing a BertForSequenceCla
ssification model from a BertForSequenceClassification model).
Device set to use cuda:0
```

In [25]:
```
results_df = pd.DataFrame(results)
results_df
```

Out[25]:

	score	token	token_str	sequence
0	0.416789	3000	paris	the capital of france is paris.
1	0.071417	22479	lille	the capital of france is lille.
2	0.063393	10241	lyon	the capital of france is lyon.
3	0.044448	16766	marseille	the capital of france is marseille.
4	0.030297	7562	tours	the capital of france is tours.

Sentence-Level Embeddings

> Note: The previous similarity score metric used static word embeddings. BERT uses contextual (dynamic) embeddings, which captures the relevance of the token in the context of the entire sentence and not just it's own word vector.

In [32]:
```
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")
```

```
bert.eval()

def mean_pooling(model_output, attention_mask):
    # model_output.last_hidden_state: [batch, seq_len, hidden]
    token_embeddings = model_output.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size
    return (token_embeddings * input_mask_expanded).sum(1) / input_mask_expanded.su

texts = [
    "Sharks are one of the oldest animals on earth.",
    "Dinosaurs were once the dominant species.",
    "My electricity went out yesterday."
]

enc = tok(texts, padding=True, truncation=True, return_tensors="pt")
with torch.no_grad():
    out = bert(**enc)

embeddings = mean_pooling(out, enc["attention_mask"])  # shape: [batch, hidden_size
embeddings = torch.nn.functional.normalize(embeddings, p=2, dim=1)  # optional L2 n

print(f"Embeddings shape: {embeddings.shape}\n")
print(f'Text 1: {texts[0]}')
print(f'Text 2: {texts[1]}')
print(f'Text 3: {texts[2]}\n')
print(f"Cosine similarity between text 1 and 2: {float(torch.matmul(embeddings[0],
print(f"Cosine similarity between text 1 and 3: {float(torch.matmul(embeddings[0],
```

```
Embeddings shape: torch.Size([3, 768])

Text 1: Sharks are one of the oldest animals on earth.
Text 2: Dinosaurs were once the dominant species.
Text 3: My electricity went out yesterday.

Cosine similarity between text 1 and 2: 0.809
Cosine similarity between text 1 and 3: 0.508
```

Sentiment Analysis

In [33]:
```
sentiment = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"  # small, fast, accurat
)
```

```
Device set to use cuda:0
```

In [37]:
```
sentences = [
    "This product is absolutely amazing!",
    "I wouldn't recommend it to anyone.",
    "It does the job, but it's nothing special."
]

sentiment_results = sentiment(sentences)
for sr, sentence in zip(sentiment_results, sentences):
    sr['sentence'] = sentence
```

```
sentiment_df = pd.DataFrame(sentiment_results)
sentiment_df.head()
```

Out[37]:

	label	score	sentence
0	POSITIVE	0.999884	This product is absolutely amazing!
1	NEGATIVE	0.798015	I wouldn't recommend it to anyone.
2	NEGATIVE	0.995147	It does the job, but it's nothing special.